

Data Lineage Estático con IA: Reconstrucción Automática y Continua del Linaje de Datos a Nivel Columna mediante Análisis de Código Fuente

Shanon Samora

Profesora: Ana Darcacha – Universidad de Palermo

Resumen-- En las organizaciones data-intensive, los datos atraviesan decenas de transformaciones antes de llegar a reportes y dashboards. Con el tiempo, este flujo se vuelve tan complejo que nadie puede responder dónde originan los números críticos. El Data Lineage resuelve esto rastreando el origen de cada dato a nivel columna. Este trabajo propone un sistema de análisis estático de código fuente que reconstruye automáticamente el linaje de datos y lo mantiene actualizado de forma continua a través de integración con repositorios Git. El enfoque híbrido combina un parser determinístico (sqlglot) para SQL estándar con un modelo de lenguaje (LLM) que interpreta casos complejos: queries dinámicas, transformaciones en pandas/PySpark y procedimientos almacenados. El grafo de linaje se mantiene en memoria del proceso del servidor y se expone a través de una interfaz web y API REST, sin requerir bases de datos externas. La validación cuantitativa sobre un repositorio sintético demuestra que el enfoque híbrido aporta un 29 % más de aristas que el parsing determinístico solo, con un 100 % de precisión sobre las aristas inferidas por LLM y un 100 % sobre las determinísticas, resolviendo el vacío tecnológico entre herramientas comerciales inaccesibles y soluciones open-source incompletas.

Palabras clave--: data lineage, análisis estático de código, inteligencia artificial, cumplimiento regulatorio, trazabilidad de datos, bases de datos de grafos, SQL, Python, BCBS 239, SOX.

I. INTRODUCCIÓN

En las organizaciones modernas, los datos son un activo crítico. Sin embargo, comprender el origen de un dato específico — por ejemplo, cómo se calcula un KPI regulatorio — se ha convertido en una tarea casi imposible. Los datos pasan por docenas de transformaciones en múltiples etapas: se extraen desde fuentes heterogéneas, se limpian en procesos intermedios, se combinan con otros datasets, se reprocesan con lógica de negocio compleja y finalmente llegan a un reporte o dashboard. A medida que los pipelines de datos crecen en volumen y complejidad, surge una pregunta fundamental que nadie en la organización puede responder con certeza: ¿de dónde proviene realmente este número?

El concepto de Data Lineage (linaje de datos) apunta a resolver exactamente este problema. Se define como la capacidad de rastrear la procedencia de un dato, identificar todos los pasos de

transformación que sufrió a lo largo de su ciclo de vida y visualizar la cadena completa de origen hasta su presentación final. En la industria financiera, esta trazabilidad no es una conveniencia técnica: es una obligación regulatoria. Normativas como BCBS 239 (principios de agregación de datos de riesgo del Comité de Basilea), SOX (Sarbanes-Oxley Act) y MiFID II (Markets in Financial Instruments Directive) exigen explícitamente que las instituciones financieras demuestren la procedencia de los datos utilizados en reportes regulatorios. El incumplimiento genera multas de millones de dólares, sanciones administrativas e incluso pérdida de licencia operativa.

Hoy, las organizaciones resuelven esta obligación de dos modos insatisfactorios. El primero es documentación manual: los equipos crean spreadsheets o wikis que describen dónde originan los datos y cómo se transforman. El problema es que esta documentación inevitablemente se desactualiza: cada cambio en el código requiere actualizar manualmente la documentación, lo que nunca ocurre consistentemente. El segundo modo es adquirir herramientas comerciales complejas — Collibra, Atlan, Informatica — que automaticen la trazabilidad. Estas soluciones funcionan bien pero cuestan entre USD 100.000 y USD 500.000 anuales, disponible solo para grandes empresas con presupuestos de TI significativos.

Entre estos dos extremos existe un vacío tecnológico que este proyecto propone ocupar.

II. JUSTIFICACIÓN

Tres factores hacen que este proyecto sea relevante y oportuno:

A. Obligación regulatoria sin solución accesible

Regulaciones como BCBS 239, SOX y MiFID II exigen que las instituciones financieras demuestren la procedencia de los datos utilizados en reportes regulatorios. El incumplimiento genera multas de millones de dólares, sanciones administrativas e incluso pérdida de licencia operativa. Hoy, las organizaciones resuelven esta obligación con documentación manual — proceso lento, propenso a errores y que se desactualiza con cada cambio de código. Las herramientas comerciales que

automaticen la trazabilidad a nivel columna (Collibra, Atlan, Informatica, Governance Catalog) tienen costos entre USD 100.000 y USD 500.000 anuales, disponible solo para grandes empresas. Una solución autoalojable y de bajo costo resolvería una necesidad urgente que millones de equipos de datos en el mundo enfrentan hoy.

B. Vacío tecnológico concreto

Existe una brecha entre las herramientas open-source (OpenLineage, Marquez, dbt) que solo cubren lineage en tiempo de ejecución, y las comerciales que cubren lineage estático pero a costos prohibitivos. El problema es que el código legacy — scripts SQL, notebooks de Jupyter, transformaciones en Python con queries dinámicas, procedimientos almacenados — representa la mayoría del código real de una organización, pero queda completamente invisible para las herramientas de lineage runtime. Ninguna solución open-source combina parsing determinístico con interpretación por IA para extraer linaje de código fuente sin ejecución. Este proyecto apunta a cubrir ese espacio intermedio.

C. Valor de negocio tangible más allá del compliance

El linaje continuo habilita tres capacidades inmediatas por las cuales una organización invertiría: (a) análisis de impacto automatizado — antes de modificar una columna en una tabla fuente, saber instantáneamente qué reportes, dashboards y pipelines downstream se ven afectados, reduciendo incidentes por cambios no evaluados; (b) reducción del tiempo de debugging — cuando un KPI muestra un valor incorrecto, trazar la cadena completa de origen en segundos en lugar de horas de investigación manual; y (c) documentación siempre actualizada — el linaje se regenera con cada cambio de código, eliminando la necesidad de mantener documentación manual que inevitablemente se desactualiza.

III. MARCO TEÓRICO

A. Data lineage y gestión de metadatos

El data lineage es la trazabilidad del recorrido que sigue un dato desde su origen hasta su uso final, incluyendo los procesos que lo crean, lo transforman y lo consumen. En entornos de gobierno de datos, esta trazabilidad se considera parte de la gestión de metadatos, porque los metadatos describen qué es cada activo de información, de dónde proviene y cómo debe interpretarse. En este trabajo, el foco está en el lineage a nivel columna, ya que ese nivel de granularidad permite identificar dependencias precisas entre campos, evaluar el impacto de cambios y respaldar auditorías con mayor exactitud [1][2].

B. Análisis estático de código fuente

El análisis estático de código es una técnica que examina el código fuente de un programa sin ejecutarlo, con el objetivo de extraer información sobre su estructura, dependencias o comportamiento (ej: puntos de transformación en SQL, Python o notebooks). Es especialmente valioso para extraer linaje de código legacy o de scripts que no pueden ejecutarse en entornos controlados de análisis, cubriendo así la mayoría del código real

que existe en una organización. A diferencia del análisis en tiempo de ejecución (o runtime), que requiere correr el programa para observar su ejecución real, el análisis estático opera directamente sobre el código. Su principal ventaja es la escalabilidad para auditar repositorios completos; su principal desafío es resolver construcciones dinámicas y ambigüedades contextuales [3].

Una técnica central del análisis estático es el parsing: la construcción de un árbol de sintaxis abstracta (Abstract Syntax Tree, AST) que representa la estructura gramatical del código y permite recorrer y consultar sus componentes —sentencias, expresiones, referencias a tablas y columnas— de forma programática [3].

C. Granularidad de lineage: nivel tabla y nivel columna

La granularidad del lineage puede ser a nivel tabla o a nivel columna. El nivel tabla identifica dependencias generales entre datasets, pero no explica cómo se calcula un campo específico. El nivel columna vincula cada atributo de salida con sus atributos de entrada y con la lógica aplicada, lo que habilita explicabilidad fina en métricas regulatorias, debugging de KPIs y validación técnica de reportes [4].

D. Lineage en tiempo de ejecución

El lineage runtime se obtiene observando ejecuciones de pipelines y jobs, por lo que captura con precisión lo que efectivamente corrió. El lineage estático se reconstruye analizando código fuente y artefactos de transformación sin ejecutar procesos. Ambos enfoques son complementarios: el runtime ofrece evidencia operacional, mientras el estático amplía cobertura sobre código no instrumentado y permite análisis preventivo antes del despliegue [4].

E. Cumplimiento regulatorio en datos financieros

En el sector financiero, la trazabilidad no es solo una buena práctica técnica, sino una exigencia normativa. BCBS 239 establece principios de agregación y reporte de datos de riesgo con foco en exactitud, integridad y trazabilidad. SOX exige controles internos robustos sobre información que impacta reportes corporativos. MiFID II impone obligaciones de registro y transparencia sobre datos vinculados a mercados e instrumentos financieros. En conjunto, estas normas justifican soluciones de lineage verificable y auditable como capacidad estructural de cumplimiento [5][6][7].

F. Representación del lineage en bases de datos orientadas a grafos

Una base de datos de grafos modela entidades como nodos y relaciones como aristas, esquema adecuado para representar dependencia entre tablas, columnas, procesos y artefactos de código. En ese modelo, el lineage se consulta mediante recorridos, ancestros, descendientes e impacto transitivo, operaciones que suelen ser costosas en enfoques puramente relacionales. Por eso los grafos ofrecen una base natural para visualización explicable y análisis de impacto en tiempo de consulta [8].

G. Herramientas de data lineage y gobierno de metadatos

OpenLineage es un estándar abierto para registrar metadatos de ejecución en pipelines de datos. Define un modelo común de eventos (job, dataset de entrada y salida, estado de ejecución) para que distintas herramientas puedan intercambiar información de lineage runtime de forma interoperable [9].

Marquez es una plataforma open-source que implementa OpenLineage para almacenar y consultar esos eventos. Permite visualizar la traza de ejecución entre jobs y datasets, y funciona como backend de metadatos para observabilidad y auditoría técnica [10].

dbt es una herramienta de transformación analítica en SQL que organiza modelos en un DAG de dependencias. Ese grafo permite entender qué modelos dependen de cuáles y exponer lineage dentro del proyecto dbt, especialmente en entornos de analytics engineering [11].

Collibra es una plataforma empresarial de gobierno de datos que integra catálogo, glosario, políticas y lineage. Su enfoque está orientado a cumplimiento, trazabilidad organizacional y análisis de impacto sobre activos de datos críticos [12].

Atlan es una plataforma de catálogo y gobernanza basada en metadatos activos. Centraliza información técnica y de negocio, y ofrece lineage para facilitar colaboración entre equipos de datos, documentación y evaluación de impacto ante cambios [13].

Informatica ofrece capacidades de lineage dentro de su ecosistema de integración, calidad y gobierno de datos. Su propuesta apunta a trazabilidad de extremo a extremo entre fuentes, transformaciones y consumos analíticos en contextos enterprise [14].

Governance Catalog refiere al componente de catálogo de gobierno en plataformas de data governance. Su función es unificar metadatos, clasificación y relaciones de dependencia para que el lineage sea útil no solo para perfiles técnicos, sino también para auditoría, riesgo y cumplimiento (por ejemplo, en Microsoft Purview) [15].

H. Modelos de lenguaje (LLM) e inteligencia artificial para comprensión de código

Los modelos de lenguaje de gran escala (Large Language Models, LLM) son redes neuronales basadas en la arquitectura Transformer [22], entrenadas sobre grandes corpus de texto y código para predecir secuencias de tokens y generar salidas a partir de instrucciones en lenguaje natural (prompting). Su capacidad para modelar dependencias contextuales de largo alcance les permite interpretar la semántica de fragmentos de código aun cuando este es sintácticamente irregular, dinámico o está parcialmente implícito. En el análisis de código fuente, los LLM se aplican a tareas de comprensión, resumen y extracción estructurada de información que los analizadores determinísticos no resuelven con facilidad, como la interpretación de SQL dinámico construido por concatenación de strings o de transformaciones escritas en pandas/PySpark [23]. A diferencia del parsing determinístico —cuyo resultado es exacto y reproducible—, la inferencia por LLM es de naturaleza probabilística: puede producir errores o "alucinaciones", por lo que conviene acotar su salida (por ejemplo, exigiendo un formato JSON estructurado y empleando temperatura baja) y registrar un nivel de confianza que la

distinga de la información extraída sintácticamente. Esta complementariedad entre la precisión del análisis estático y la comprensión semántica del modelo de lenguaje constituye el fundamento del enfoque híbrido propuesto en este trabajo.

I. Orquestación de datos y pipelines ETL

En las arquitecturas de datos modernas, la extracción, transformación y carga de información se organiza en pipelines de ETL/ELT (Extract, Transform, Load) coordinados por herramientas de orquestación. Un orquestador define y ejecuta, en un orden determinado por sus dependencias, actividades tales como la copia de datos entre almacenes, la ejecución de transformaciones o la invocación de procedimientos y notebooks [4]. Plataformas como Azure Data Factory (ADF) [21] y Apache Airflow [25] representan estos flujos como artefactos versionables —definiciones JSON de pipelines, datasets y dataflows en el caso de ADF; grafos dirigidos acíclicos (DAG) escritos en Python en el caso de Airflow—, que describen tanto los movimientos físicos de datos como las cadenas de orquestación entre componentes heterogéneos. Estos artefactos son una fuente de linaje de primer orden —a menudo la única evidencia del flujo de datos a nivel de plataforma—, pero quedan fuera del alcance de los parsers SQL convencionales, lo que justifica un análisis especializado de su estructura para reconstruir el linaje de extremo a extremo.

J. Integración continua (CI/CD) y control de versiones

El control de versiones —con Git como estándar de facto— registra la evolución del código fuente y habilita la colaboración mediante ramas y pull requests, las unidades de cambio que se revisan antes de integrarse a la rama principal. Sobre esa base, las prácticas de integración y entrega continuas (CI/CD, Continuous Integration / Continuous Delivery) automatizan la validación de cada cambio: ante cada commit o pull request se ejecutan automáticamente pruebas, análisis y controles de calidad cuyos resultados se devuelven al desarrollador antes del merge [24]. Plataformas como GitHub Actions [19] permiten declarar estos flujos como workflows disparados por eventos del repositorio (por ejemplo, la apertura o actualización de un pull request), apoyándose en bibliotecas de acceso al historial como GitPython [20]. Incorporar el análisis de linaje en esta etapa del ciclo de desarrollo transforma la trazabilidad de un artefacto generado puntualmente en una capacidad continua: el linaje se recalcula con cada cambio y su impacto potencial se expone a los revisores antes de que el código llegue a producción, mitigando la desactualización que afecta a la documentación manual.

K. Análisis de impacto

El análisis de impacto es la operación complementaria a la trazabilidad de procedencia: dado un activo de datos (una tabla o una columna), consiste en identificar todos los activos que dependen de él aguas abajo (downstream) y que, por lo tanto, se verían afectados ante una modificación o eliminación. Sobre un grafo de linaje se resuelve como un recorrido de descendientes a partir del nodo modificado. Esta capacidad sustenta la evaluación preventiva de cambios —antes del despliegue— y resulta especialmente valiosa en contextos regulatorios, donde un cambio no evaluado en una columna

fuerza puede propagarse de forma silenciosa hasta un reporte de cumplimiento [4].

IV. ESTADO DEL ARTE

El Data Lineage se consolidó como una capacidad central en arquitecturas modernas de datos, ya que permite identificar el origen, las transformaciones y el consumo de la información a lo largo de pipelines analíticos y regulatorios. En términos prácticos, su valor no se limita a la documentación técnica: también habilita análisis de impacto, auditoría, cumplimiento normativo y resolución más rápida de incidentes en entornos donde los datos se transforman en múltiples etapas antes de llegar a reportes o dashboards.

Los enfoques existentes pueden agruparse, en primer lugar, en soluciones de observación en runtime. Herramientas como OpenLineage, Marquez y distintos conectores de orquestación capturan eventos durante la ejecución de procesos, lo que permite obtener trazabilidad operativa en tiempo real. Sin embargo, su cobertura depende de que los pipelines estén instrumentados. Esto deja fuera una parte importante del ecosistema real de una organización: scripts legacy, notebooks ejecutados manualmente, procesos ad hoc y transformaciones dispersas en Python o PySpark.

Una segunda familia está compuesta por plataformas de gobierno y catálogo de datos, como Collibra, Atlan, Informatica o Microsoft Purview. Estas soluciones integran metadatos, catálogo y visualización de lineage, y suelen ofrecer capacidades útiles para trazabilidad e impacto de cambios. No obstante, su adopción suele verse limitada por el costo, la complejidad de implementación y la dependencia de ecosistemas propietarios, lo que reduce su accesibilidad para equipos medianos o proyectos que requieren una alternativa autoalojable.

En paralelo, existen propuestas de lineage estático basadas en parsers SQL y análisis de dependencias del código fuente. Este enfoque mejora la cobertura sobre repositorios sin necesidad de ejecutar pipelines, pero presenta límites cuando aparecen SQL dinámico, macros, procedimientos almacenados poco documentados o transformaciones escritas en Python/PySpark. En esos casos, la extracción del linaje a nivel columna pierde precisión justamente en los escenarios que más valor tienen para auditoría y análisis de impacto.

Más recientemente, comenzaron a explorarse modelos de lenguaje (LLM) para comprensión de código, extracción semántica y asistencia en documentación técnica. Estos avances abren la posibilidad de interpretar patrones que los parsers determinísticos no resuelven con facilidad, especialmente en lógica compleja o parcialmente implícita. En paralelo, las bases de datos orientadas a grafos —como Neo4j— se consolidaron como una representación adecuada para modelar relaciones entre tablas, columnas, transformaciones y activos de datos, facilitando consultas de recorrido e impacto con mayor expresividad.

A partir de este relevamiento, se identifica un vacío concreto: no se encontró una solución accesible que combine de forma robusta análisis estático multi-lenguaje, lineage a nivel columna, cobertura de casos no instrumentados y actualización

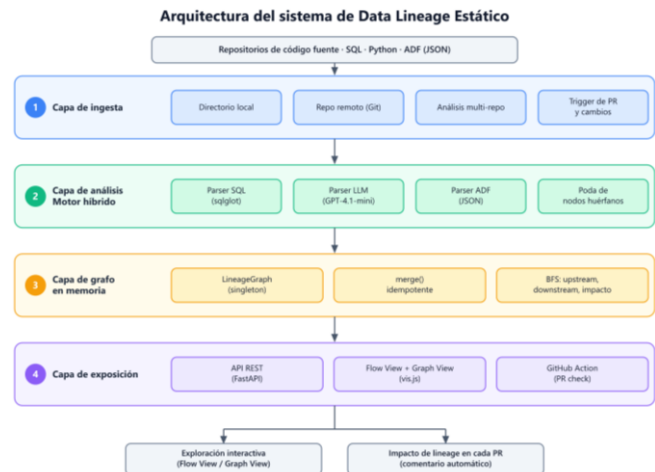
continua integrada al flujo de desarrollo. En ese espacio se posiciona este trabajo, proponiendo un enfoque híbrido que combina parsing determinístico e interpretación asistida por IA para mejorar la cobertura y la precisión sin depender de la ejecución en producción.

V. DESARROLLO

La solución propuesta se diseña como una aplicación que se conecta al ciclo de desarrollo de software, con capacidad para analizar uno o varios repositorios de código y reconstruir el linaje de datos de forma continua. A diferencia de una herramienta que se ejecuta una sola vez sobre un proyecto cerrado, este sistema se integra con GitHub para revisar cambios en cada pull request, actualizar el grafo de lineage y exponer los impactos potenciales antes de que el código llegue a producción. De este modo, el linaje deja de ser un artefacto estático y pasa a funcionar como una capacidad operativa del proceso de desarrollo.

A. Arquitectura general

La arquitectura se organiza en cuatro capas: ingesta, análisis, grafo en memoria y exposición.



La capa de ingesta se encarga de obtener el contenido de los repositorios, identificar archivos relevantes (SQL, Python, JSON de Azure Data Factory) y detectar cuándo ocurre un cambio sobre una rama o un pull request. El sistema soporta análisis local de directorios, análisis de repositorios remotos clonados vía Git y análisis multi-repo donde se combinan grafos de distintos orígenes.

La capa de análisis procesa cada archivo detectado mediante un motor híbrido que decide, según el tipo de archivo y la complejidad del código, si aplica parsing determinístico o interpretación por LLM. El resultado de cada análisis es un fragmento del grafo de lineage (nodos y aristas) que se fusiona con el grafo global.

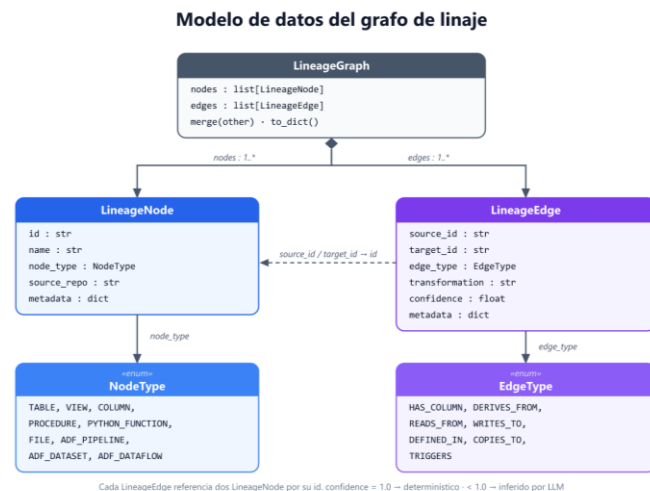
La capa de grafo en memoria mantiene la estructura del lineage como un singleton del proceso del servidor (`_graph: LineageGraph` en `src/api.py`). Los recorridos de ancestros, descendientes e impacto se resuelven con un BFS sobre listas de adyacencia construidas al vuelo, distinguiendo aristas "hacia atrás" (consumer → producer: `'DERIVES_FROM'`,

'READS_FROM') de aristas "hacia adelante" (producir → consumer: 'WRITES_TO', 'COPIES_TO') y normalizando la dirección para que la lógica de BFS sea uniforme. Esta decisión simplifica el despliegue —no requiere bases de datos externas ni infraestructura adicional— a costa de perder persistencia entre reinicios; un trade-off adecuado para el alcance de POC y fácilmente extensible a un backend persistente.

La capa de exposición presenta el lineage a través de una API REST (FastAPI) con endpoints para análisis, consulta de grafo, búsqueda, impacto y validación de pull requests, y una interfaz web (Flow View estilo Databricks Unity Catalog + Graph View con vis.js) para visualización interactiva del grafo.

B. Modelo de datos

El grafo de lineage se modela con dos primitivas: nodos (LineageNode) y aristas (LineageEdge).



Cada nodo tiene un identificador único, un nombre descriptivo, un tipo (NodeType) y un campo opcional source_repo que indica de qué repositorio proviene. Los tipos de nodo soportados son: TABLE, VIEW, COLUMN, PROCEDURE, PYTHON_FUNCTION, FILE, ADF_PIPELINE, ADF_DATASET y ADF_DATAFLOW. Esta taxonomía cubre activos SQL clásicos, artefactos de código Python y componentes de pipelines de Azure Data Factory.

Cada arista conecta un nodo origen con un nodo destino y tiene un tipo (EdgeType), una descripción opcional de la transformación aplicada y un nivel de confianza (confidence) que indica si la relación fue extraída por parsing determinístico (1.0) o por interpretación LLM (típicamente 0.7–0.9). Los tipos de arista son: HAS_COLUMN, DERIVES_FROM, READS_FROM, WRITES_TO, DEFINED_IN, COPIES_TO y TRIGGERS.

El grafo completo (LineageGraph) expone métodos para agregar nodos y aristas, fusionar grafos parciales (merge) y serializar a diccionario. La fusión es idempotente: si un nodo o arista ya existe, no se duplica.

C. Motor de análisis híbrido

El componente central del sistema es el motor de análisis ('src/engine.py'), que orquesta la elección del parser adecuado para cada archivo. El flujo de decisión es el siguiente:

1. Se recibe un archivo con su ruta y contenido.
2. Se detecta el tipo de archivo: .sql/.ddl/.dml/.hql se dirigen al parser SQL, .py/.pyspark al parser Python/LLM, y archivos JSON en carpetas pipeline/, dataset/ o dataflow/ al parser de Azure Data Factory.
3. Para archivos SQL, se intenta primero el parser determinístico (sqlglot). Luego se evalúa si es necesario un fallback al LLM mediante un conjunto de expresiones regulares con word boundaries que detectan: CREATE PROCEDURE/FUNCTION, LANGUAGE plpgsql, EXECUTE IMMEDIATE, sp_executesql, y bloques procedurales (BEGIN seguido de DECLARE/SET/EXEC). El diseño basado en regex evita falsos positivos comunes —como el END de un CASE WHEN ... END o un BEGIN dentro de un comentario— que en versiones anteriores disparaban llamadas innecesarias al LLM. Adicionalmente, si el parser determinístico produce cero aristas de linaje sobre un archivo no trivial (>100 caracteres), se asume que el contenido es opaco al análisis sintáctico y se delega al LLM.
4. Para archivos Python, se invoca siempre el parser LLM porque las transformaciones en pandas, PySpark o SQLAlchemy requieren interpretación semántica.
5. Para archivos ADF, se usa un parser determinístico especializado que interpreta la estructura JSON de pipelines, datasets y dataflows.

Tras el análisis de todos los archivos, el motor ejecuta una pasada de poda ('_prune_dead_nodes') que elimina nodos "huérfanos" creados por el LLM: variables locales de pandas que el modelo identifica como entidades pero que solo tienen una arista 'DEFINED_IN' al archivo, sin participar en flujos de datos reales. Esta limpieza mantiene el grafo enfocado en entidades persistidas y relaciones de lineage genuinas.

Este enfoque híbrido maximiza la precisión en casos estándar (parsing determinístico) y extiende la cobertura a casos complejos (LLM), registrando el nivel de confianza de cada relación extraída.

D. Parser determinístico SQL

El parser SQL (parser_sql.py) utiliza la librería sqlglot [16] para construir un AST (Abstract Syntax Tree) del código SQL y extraer relaciones de lineage a nivel columna. Su cobertura incluye: sentencias SELECT con joins, subqueries y CTEs, que permiten identificar qué columnas de qué tablas alimentan cada columna de salida; sentencias CREATE TABLE ... AS SELECT e INSERT INTO ... SELECT, que detectan la tabla destino y las columnas fuente; definiciones de vistas (CREATE VIEW), que modelan la vista como nodo y vinculan sus columnas a las tablas subyacentes; procedimientos almacenados (CREATE PROCEDURE), de los cuales se extrae el cuerpo y se analizan las sentencias internas como SQL independiente; y transformaciones con funciones (COALESCE, CASE, agregaciones), cuya expresión se registra en la arista correspondiente.

El parser procesa archivos con múltiples sentencias separadas por punto y coma, lo que permite analizar scripts de migración y pipelines SQL completos en una sola pasada.

E. Parser LLM para Python y SQL complejo

El parser LLM (parser_llm.py) envía el código fuente a un modelo de lenguaje (GPT-4.1-mini vía API de OpenAI [17]) con un prompt estructurado que solicita la extracción de nodos y aristas en formato JSON. El prompt incluye instrucciones para identificar tablas, columnas y funciones referenciadas en el código; detectar operaciones de lectura (pd.read_sql, spark.read, SELECT) y escritura (to_sql, INSERT, saveAsTable); extraer relaciones DERIVES_FROM a nivel columna cuando la lógica lo permite; y devolver el resultado en un schema predefinido que el sistema parsea y convierte a LineageGraph.

Se utiliza cuando el parser determinístico no puede resolver el contenido: SQL dinámico construido con concatenación de strings, transformaciones en pandas/PySpark, notebooks con lógica mixta y procedimientos almacenados con EXEC dinámico.

F. Parser de Azure Data Factory

El parser ADF (src/parser_adf.py) interpreta los archivos JSON que Azure Data Factory exporta al repositorio Git cuando se configura integración con control de versiones. Soporta tres tipos de artefacto y cinco tipos de actividad:

- Pipelines: se extraen las actividades.
- 'Copy': genera aristas 'COPIES_TO' entre el dataset de entrada y el de salida, y aristas 'READS_FROM'/'WRITES_TO' desde el pipeline a cada dataset.
- 'ExecutePipeline': genera aristas 'TRIGGERS' entre el pipeline padre y el hijo, modelando dependencias de orquestación.
- 'MappingDataFlow' / 'ExecuteDataFlow': referencian el dataflow correspondiente con una arista 'TRIGGERS'.
- 'DatabricksNotebook' / 'DatabricksPythonActivity' / 'AzureDatabricksActivity': el parser extrae el 'notebookPath' de las propiedades de la actividad y genera una arista 'TRIGGERS' desde el pipeline a un nodo 'FILE' cuyo ID coincide con la ruta del notebook. Si esa ruta es analizada también por el parser Python/LLM, ambos nodos se fusionan por ID — habilitando linaje cross-platform entre la orquestación de ADF y el código Python.
- 'SqlServerStoredProcedure' / 'StoredProcedure': el parser extrae el nombre del procedimiento (eliminando el prefijo de schema, si lo hay) y genera una arista 'TRIGGERS' desde el pipeline al nodo 'PROCEDURE' correspondiente. Si el procedimiento fue detectado en paralelo por el parser SQL/LLM al analizar el archivo '.sql' que lo define, los nodos se fusionan automáticamente.
- Datasets: se crean nodos 'ADF_DATASET' con metadata sobre la estructura (tipo de almacenamiento, nombre lógico). Si el dataset declara una tabla física subyacente ('tableName' o 'schema.table'), se crea un nodo 'TABLE' adicional con una

arista de aliasing bidireccional, permitiendo que el grafo refleje correctamente sinks ADF que escriben a tablas SQL existentes.

- Dataflows: se analizan las transformaciones 'sources' y 'sinks' para generar aristas que conectan los datasets de origen con los de destino a través del dataflow como nodo intermedio.

Esta cobertura permite incluir en el grafo de lineage tanto los movimientos de datos físicos (Copy entre Blob, SQL, Parquet) como las cadenas de orquestación cruzando lenguajes y plataformas (ADF → Notebook Python → procedimientos SQL), completando la visión end-to-end del flujo de datos en organizaciones que utilizan ADF como orquestador.

G. Análisis multi-repositorio

En entornos reales, el código de datos de una organización no vive en un solo repositorio. Es común tener un repositorio para SQL (DDL, stored procedures), otro para transformaciones Python (ETL, notebooks) y otro para pipelines de orquestación (ADF, Airflow). El sistema soporta análisis multi-repo mediante la siguiente mecánica:

1. Se define una lista de fuentes (RepoSource) en la configuración, cada una con una URL de repositorio y una rama.
2. El motor clona cada repositorio en un directorio temporal.
3. Se ejecuta el análisis estándar sobre cada repositorio, etiquetando cada nodo con su source_repo de origen.
4. Los grafos parciales se fusionan en un grafo global mediante el método merge() de LineageGraph.

Este enfoque permite, por ejemplo, descubrir que una tabla creada en el repositorio SQL es leída por un pipeline en el repositorio ADF y transformada por un script en el repositorio Python, reconstruyendo la cadena completa de lineage cross-repo.

H. Almacenamiento en memoria

El grafo de lineage se mantiene en una estructura 'LineageGraph' (Pydantic) viva en el proceso del servidor a través del módulo 'src/api.py'. El método 'LineageGraph.merge()' garantiza idempotencia: si un nodo o arista ya existe, no se duplica. Adicionalmente, cuando un nodo es detectado simultáneamente por el parser determinístico y por el LLM (típicamente tablas de staging que el LLM también identifica al analizar código Python), la política de merge preserva la versión determinística por sobre la versión LLM, evitando que metadatos exactos sean reemplazados por inferencias probabilísticas.

Los recorridos de ancestros (upstream), descendientes (downstream) e impacto transitivo se implementan como BFS sobre listas de adyacencia construidas al vuelo en 'src/api.py: build_adjacency()'. La función discrimina entre dos tipos de aristas para normalizar la semántica de la dirección:

1. Aristas "hacia atrás" ('DERIVES_FROM', 'READS_FROM'): apuntan desde el consumer al producer ('output_col DERIVES_FROM input_col'). El BFS las atraviesa "hacia adelante" para alcanzar los orígenes.

2. Aristas "hacia adelante" ('WRITES_TO', 'COPIES_TO'): apuntan desde el productor al consumer ('pipeline WRITES_TO dataset'). El BFS las invierte al construir la adyacencia para que la lógica de recorrido sea uniforme.

Adicionalmente, las aristas "dataset ADF ↔ tabla física" se tratan como bidireccionales, dado que el dataset ADF y la tabla SQL subyacente representan la misma entidad lógica: una conecta a la otra independientemente del sentido del flujo de datos en cada Copy activity.

Esta capacidad es la base del análisis de impacto: cuando se modifica una tabla o columna, el sistema recorre el grafo downstream para identificar todos los activos afectados, atravesando capas SQL, Python y ADF de forma uniforme.

I. Integración CI/CD con GitHub Actions

Una de las contribuciones principales de este trabajo es la integración del análisis de lineage directamente en el flujo de CI/CD. El sistema incluye un GitHub Action (lineage-check.yml) que se activa automáticamente cuando se abre o actualiza un pull request. El flujo es:

1. Checkout: se clona el repositorio con historial completo (fetch-depth: 0) para poder comparar el commit del PR con el commit base.
2. Análisis before: el orquestador ('src/pr_analyzer.py') materializa el commit base en un 'git worktree' temporal y ejecuta el motor de lineage sobre cada uno de los tres repositorios fuente (SQL, Python y ADF) en ese estado, obteniendo el grafo previo al cambio.
3. Análisis after: se ejecuta el motor sobre el commit del PR (head), también sobre los tres repositorios, obteniendo el grafo posterior al cambio.
4. Diff e impacto: el módulo 'diff.py' compara ambos grafos, identifica nodos y aristas agregados, eliminados y modificados, y ejecuta un recorrido BFS downstream para determinar todos los activos impactados. Como el análisis abarca los tres repositorios de forma conjunta, un cambio en un repositorio (por ejemplo, Python) puede revelar impacto en otro (SQL o ADF), materializando la detección de impacto cross-repo dentro del CI.
5. Comentario en el PR: el resultado se formatea como un reporte Markdown y se publica como comentario en el pull request, indicando cantidad de cambios en el grafo, nodos impactados, nivel de riesgo y detalle de cada cambio. La publicación es idempotente: el flujo busca un comentario previo del bot y lo actualiza en lugar de duplicarlo. Además, el paso de análisis está marcado como 'continue-on-error', de modo que una falla transitoria (timeout de la API del LLM, archivo malformado) nunca bloquea el PR: en ese caso se publica un reporte degradado que explica la causa probable.

The screenshot shows a GitHub Pull Request comment from the 'github-actions' bot. The comment is titled 'Data Lineage Impact Report' and includes a summary of changes and a detailed list of downstream impacts.

Data Lineage Impact Report

15 downstream nodes may be affected by changes to 1 file.

Metric	Count
Nodes added	0
Nodes removed	0
Edges added	0
Edges removed	0
Edges modified	3
Downstream nodes impacted	15

Files analysed

- sample_repo_sql/02_intermediate_views.sql

Edge Changes

Status	Source	Target	Relationship
MODIFIED	int_daily_balances.daily_net_amount_usd	stg_transactions.transaction_type	derives from

Downstream Impact

The following nodes are downstream of the changes and may be affected:

- int_python_customer_scores.balance_volatility_usd
- int_python_customer_scores.risk_score
- int_python_customer_scores.segment
- int_python_customer_scores.total_net_usd
- rpt_customer_exposure.cumulative_exposure_usd
- rpt_customer_exposure.daily_net_amount_usd
- rpt_customer_exposure.python_risk_score
- rpt_customer_exposure.python_risk_segment
- rpt_customer_scorecard.balance_volatility_usd
- rpt_customer_scorecard.risk_score
- rpt_customer_scorecard.segment
- rpt_customer_scorecard.total_net_usd
- rpt_monthly_summary.net_position_usd
- rpt_regional_risk_summary.avg_python_risk_score
- rpt_regional_risk_summary.python_risk_segment

Generated by [Data Lineage POC](#) — hybrid static analysis (deterministic SQL parser + LLM fallback).

Este flujo permite que los revisores del PR vean el impacto de lineage antes de aprobar el merge, reduciendo el riesgo de cambios no evaluados que afecten reportes o pipelines downstream.

J. Motor de diff y análisis de impacto

El módulo diff.py implementa la comparación entre dos grafos de lineage (before y after). El algoritmo:

1. Compara nodos por id: los presentes solo en after son ADDED, los presentes solo en before son REMOVED, los presentes en ambos pero con atributos diferentes son MODIFIED.
2. Compara aristas por (source_id, target_id, edge_type): misma lógica de ADDED/REMOVED/MODIFIED.

3. Para cada nodo o arista con cambios, ejecuta un recorrido BFS en el grafo after para recolectar todos los nodos downstream impactados.
4. Genera un LineageDiff con la lista de cambios y el conjunto de nodos impactados.

El resultado se expone tanto como estructura de datos (para la API) como reporte Markdown (para el comentario en GitHub). La orquestación de extremo a extremo —materialización de los estados base y head en un 'git worktree', análisis multi-repo y diff— está encapsulada en 'src/pr_analyzer.py', que reutiliza 'diff.py' como motor de comparación.

K. API REST

La API (api.py) expone los siguientes endpoints principales a través de FastAPI:

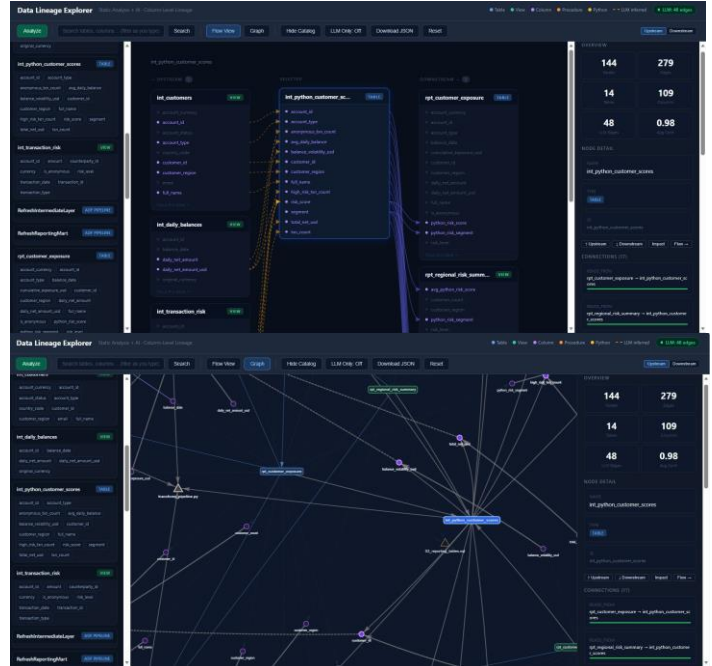
- POST /analyze: recibe una ruta local y ejecuta el análisis completo, retornando el grafo de lineage.
- POST /analyze-multi: recibe una lista de fuentes de repositorio y ejecuta análisis multi-repo.
- GET /graph: retorna el grafo completo almacenado en memoria.
- GET /upstream/{node_id} y GET /downstream/{node_id}: retornan los ancestros o descendientes de un nodo.
- GET /impact/{node_id}: retorna el análisis de impacto (nodos afectados downstream).
- GET /search?q=...: busca nodos por nombre o tipo.
- POST /pr-check: recibe las referencias Git base y head de un pull request junto con la lista de repositorios fuente, ejecuta el análisis de impacto de lineage multi-repo (orquestado por 'src/pr_analyzer.py') y retorna tanto la estructura de diff como un reporte Markdown.
- GET /: sirve la interfaz web (template Jinja2 single-page) que consume el resto de los endpoints para visualizar el grafo.

Todos los endpoints API devuelven JSON y están documentados automáticamente vía OpenAPI (Swagger UI, disponible en 'docs').

L. Interfaz web

El sistema incluye una interfaz web ('templates/index.html') que consume la API REST y ofrece dos modos de visualización complementarios del grafo de lineage:

- Flow View (estilo Databricks Unity Catalog): es la vista por defecto. Organiza las entidades en columnas upstream / centro / downstream y despliega el linaje a nivel columna de la entidad seleccionada, distinguiendo visualmente las aristas determinísticas de las inferidas por LLM según su nivel de confianza.
- Graph View (vis.js): renderiza el grafo completo como una red de nodos y aristas navegable, con recorrido interactivo de ancestros y descendientes.



Sobre ambas vistas, la interfaz provee una barra de búsqueda con filtrado en vivo a medida que se escribe (sobre nombres de tablas y columnas), filtros por tipo de nodo mediante chips, alternancia entre exploración upstream y downstream, y un botón de exportación del grafo completo a JSON. De este modo, la interfaz no solo visualiza el linaje sino que habilita su exploración y extracción para auditoría.

M. Validación

Para validar el sistema se diseñaron tres repositorios de ejemplo independientes —'sample_repo_sql', 'sample_repo_python' y 'sample_repo_adf'— que reproducen el escenario multi-repo descrito en la sección de análisis multi-repositorio: en una organización real, el SQL, las transformaciones Python y los pipelines de orquestación viven en repositorios separados. Cada uno contiene artefactos representativos de un entorno productivo:

- SQL ('sample_repo_sql/'): 4 archivos con DDL de staging tables, intermediate views, reporting tables y stored procedures que cubren CREATE TABLE, CREATE VIEW, INSERT INTO ... SELECT, joins, subqueries, CTEs, window functions y procedimientos almacenados con bloques PL/pgSQL.
- Python ('sample_repo_python/'): 1 archivo ('transform_pipeline.py') con un pipeline de transformación que demuestra el patrón "Python en el medio" — lee tres vistas SQL intermedias ('int_customers', 'int_daily_balances', 'int_transaction_risk') vía pandas + SQLAlchemy, computa un risk scoring compuesto y un segmento de clasificación, y persiste el resultado en una tabla SQL ('int_python_customer_scores') que el layer reporting consume aguas abajo.

- ADF ('sample_repo_adf'): 11 archivos JSON modelando un flujo end-to-end (3 pipelines encadenados, 8 datasets). El pipeline 'IngestRawData' copia archivos Blob (CSV/Parquet) a la capa de staging SQL; 'RefreshIntermediateLayer' ejecuta el notebook Python via 'DatabricksNotebook'; 'RefreshReportingMart' invoca el stored procedure via 'SqlServerStoredProcedure' y exporta el reporte final a una BI mart externa via Copy activity.

El análisis combinado de los tres repositorios produce un grafo único —fusionado mediante 'merge()' y etiquetando cada nodo con su 'source_repo'— con nodos de tipo 'TABLE', 'VIEW', 'COLUMN', 'PROCEDURE', 'PYTHON_FUNCTION', 'FILE', 'ADF_PIPELINE', 'ADF_DATASET' y aristas que conectan toda la cadena desde los archivos Blob de origen hasta el mart externo final, atravesando cinco tecnologías: ADF, SQL DDL, SQL views, Python pandas y procedimientos almacenados. De este modo, la validación ejercita simultáneamente el linaje cross-language (SQL → Python → SQL) y la reconstrucción de linaje cross-repo.

VI. CONCLUSIONES

El sistema desarrollado demuestra que es viable construir una solución de data lineage estático que combine parsing determinístico e interpretación por IA para reconstruir automáticamente el linaje de datos a nivel columna desde código fuente, sin necesidad de ejecutar los pipelines en producción.

Las principales contribuciones de este trabajo son:

1. Enfoque híbrido determinístico + LLM: el uso de sqlglot para SQL estándar garantiza precisión y velocidad en los casos resolubles por parsing, mientras que el fallback a GPT-4.1-mini extiende la cobertura a SQL dinámico, transformaciones Python y procedimientos almacenados complejos. El campo confidence en cada arista permite distinguir el origen de cada relación extraída.
2. Cobertura multi-lenguaje y multi-plataforma: el sistema analiza SQL, Python y artefactos de Azure Data Factory de forma unificada, produciendo un grafo de lineage integrado que cubre toda la cadena de datos end-to-end. El soporte de actividades 'DatabricksNotebook' y 'SqlServerStoredProcedure' en ADF permite que el grafo conecte la orquestación con el código Python y los procedimientos SQL ejecutados, materializando linaje cross-platform que ninguna herramienta open-source consultada resuelve de forma unificada. El soporte multi-repo permite combinar grafos de distintos repositorios.
3. Integración continua con el flujo de desarrollo: la incorporación del análisis de lineage como paso automático en los pull requests (via GitHub Actions) transforma el lineage de un artefacto estático a una capacidad operativa que informa decisiones de merge en tiempo real.

4. Análisis de impacto preventivo: el motor de diff y el recorrido BFS downstream permiten identificar, antes del merge, todos los activos de datos que se verían afectados por un cambio propuesto, reduciendo el riesgo de incidentes por modificaciones no evaluadas.

5. Arquitectura extensible: el diseño modular (parsers, engine, modelo de grafo en memoria, API) permite incorporar nuevos lenguajes, plataformas de orquestación o backends de almacenamiento sin modificar el núcleo del sistema.

6. Validación cuantitativa documentada: la implementación incluye una suite de 77 tests automatizados y un informe de validación ('docs/validation.md') con baselines reproducibles y revisión manual de las aristas inferidas por LLM. La precisión medida (100 % determinístico, 100 % LLM) y el incremento de cobertura cuantificado (+29 % de aristas con el enfoque híbrido) constituyen evidencia empírica de las hipótesis del trabajo.

El principal limitante del enfoque LLM es el costo por token y la latencia de las llamadas a la API, lo que hace que el análisis de repositorios muy grandes pueda ser lento o costoso. Este limitante se mitiga mediante el enfoque híbrido: el LLM solo se invoca cuando el parser determinístico no puede resolver el contenido. Un segundo limitante es que el grafo se mantiene en memoria del proceso del servidor: una caída o reinicio implica re-analizar el repositorio. Esta decisión es deliberada para el alcance del POC pero requeriría un backend persistente en un despliegue productivo.

VII. LÍNEAS FUTURAS

A partir del trabajo realizado, se identifican las siguientes líneas de evolución:

1. Soporte para dbt y Airflow: incorporar parsers para modelos dbt (análisis de archivos .sql con macros Jinja y el manifest.json) y DAGs de Apache Airflow (análisis de archivos Python con decoradores @task y dependencias entre tasks).
2. Lineage temporal: registrar versiones históricas del grafo de lineage con timestamps, permitiendo consultar cómo era el lineage de un activo en una fecha específica y comparar evolución a lo largo del tiempo.
3. Integración con catálogos de datos: conectar el grafo de lineage con plataformas de catálogo (DataHub, OpenMetadata) para enriquecer los metadatos con información de lineage automatizada.
4. Lineage de notebooks Jupyter: desarrollar un parser especializado que interprete celdas de notebooks (.ipynb), detecte dependencias entre celdas y extraiga lineage de código mixto (SQL + Python en el mismo notebook).
5. Scoring de calidad del lineage: implementar métricas de completitud y confianza del grafo (porcentaje de nodos con lineage completo, distribución de confidence scores) para que los equipos puedan evaluar la madurez de su trazabilidad.

6. Soporte para Spark SQL y transformaciones PySpark nativas: extender el parser determinístico para interpretar operaciones de DataFrame de PySpark (join, groupBy, agg, withColumn) sin necesidad de recurrir al LLM.

7. Plugin para IDE: desarrollar una extensión de Visual Studio Code que muestre el lineage de un archivo o función directamente en el editor, permitiendo a los desarrolladores consultar el impacto de sus cambios antes de hacer commit.

8. Backend de persistencia opcional: el modelo actual de grafo en memoria es apropiado para POC y demos, pero en un despliegue productivo conviene persistir el grafo en un store dedicado. Las opciones naturales son Neo4j (con consultas Cypher para recorridos), Postgres con la extensión `pg\_lineage` o `recursive CTE`, o backends document-oriented si las consultas de recorrido son más simples. El módulo de almacenamiento actual está aislado en `src/api.py` (función `\_build\_adjacency` y el singleton `\_graph`), facilitando esta extensión sin alterar parsers ni motor.

VIII. REFERENCIAS

- [1] DAMA International, The DAMA Guide to the Data Management Body of Knowledge (DAMA-DMBOK), 2^a ed. Available: <https://books.google.com.co/books?id=7FdNCwAAQBAJ&printsec=frontcover>
- [2] IBM, "Data Lineage". Available: <https://www.ibm.com/products/watsonx-data-intelligence/data-lineage>
- [3] F. Nielson, H. R. Nielson, and C. Hankin, Principles of Program Analysis. Springer, 1999. Available: https://www.researchgate.net/publication/265352570_Principles_of_Program_Analysis
- [4] R. Kimball and M. Ross, The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling, 3rd ed. Wiley, 2013. Available: <https://ia801609.us.archive.org/14/items/the-data-warehouse-toolkit-kimball/The%20Data%20Warehouse%20Toolkit%20-%20Kimball.pdf>
- [5] Basel Committee on Banking Supervision, Principles for Effective Risk Data Aggregation and Risk Reporting (BCBS 239). Bank for International Settlements, 2013. Available at: <https://www.bis.org/publ/bcbs239.pdf>
- [6] U.S. Congress, Sarbanes-Oxley Act of 2002, Public Law 107-204, 2002. Available at: https://www.dol.gov/sites/dolgov/files/oalj/PUBLIC/WHI_STLEBLOWER/REFERENCES/STATUTES/SARBANES_OXLEY_ACT_OF_2002.PDF
- [7] European Parliament and Council, Directive 2014/65/EU on Markets in Financial Instruments (MiFID II), 2014. Available at: <https://eur-lex.europa.eu/eli/dir/2014/65/oj/eng>
- [8] I. Robinson, J. Webber, and E. Eifrem, Graph Databases, 2nd ed. O'Reilly Media, 2015. Available at: https://web4.ensiie.fr/~stefania.dumbrava/OReilly_Graph_Databases.pdf
- [9] OpenLineage, "OpenLineage Documentation." Available: <https://openlineage.io/docs/>
- [10] Marquez, "Marquez Documentation." Available: <https://marquezproject.ai/docs/>
- [11] dbt Labs, "dbt Documentation." Available: <https://docs.getdbt.com/>
- [12] Collibra, "Data Lineage." Available: <https://www.collibra.com/us/en/products/data-lineage>
- [13] Atlan, "Data Lineage." Available: <https://atlan.com/data-lineage/>
- [14] Informatica, "Data Lineage." Available: <https://www.informatica.com/services-and-training/glossary-of-terms/data-lineage-definition.html>
- [15] Microsoft, "Microsoft Purview Documentation." Available: <https://learn.microsoft.com/en-us/purview/>
- [16] sqlglot, "sqlglot: Python SQL Parser and Transpiler." Available: <https://sqlglot.com/>
- [17] OpenAI, "OpenAI API Documentation." Available: <https://platform.openai.com/docs/>
- [18] FastAPI, "FastAPI Documentation." Available: <https://fastapi.tiangolo.com/>
- [19] GitHub, "GitHub Actions Documentation." Available: <https://docs.github.com/en/actions>
- [20] GitPython, "GitPython Documentation." Available: <https://gitpython.readthedocs.io/>
- [21] Microsoft, "Azure Data Factory Documentation." Available: <https://learn.microsoft.com/en-us/azure/data-factory/>
- [22] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," in Advances in Neural Information Processing Systems (NeurIPS), 2017. Available: <https://arxiv.org/abs/1706.03762>
- [23] M. Chen et al., "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021. Available: <https://arxiv.org/abs/2107.03374>
- [24] J. Humble and D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Boston: Addison-Wesley, 2010.
- [25] Apache Software Foundation, "Apache Airflow Documentation." Available: <https://airflow.apache.org/docs/>

IX. BIOGRAFÍA DEL AUTOR



Shanon Samora nació en el partido de San Martín, Provincia de Buenos Aires, Argentina, el 26 de diciembre de 2000. Es ingeniero de software, especializado en el desarrollo de pipelines escalables, automatización de procesos, optimización de performance y construcción de plataformas de datos en entornos cloud. Actualmente se desempeña como Data Engineer en CraftLabs, donde participa en el diseño y desarrollo de dominios de datos compuestos por múltiples pipelines orquestados, integrando repositorios en GitHub con despliegues automatizados mediante Azure DevOps y ejecución en Azure Databricks. Su trabajo incluye el procesamiento y movimiento de grandes

volúmenes de información en arquitecturas basadas en Azure Data Lake, Snowflake, SQL Server y Denodo, así como la gestión de controles de acceso y autorizaciones para garantizar integraciones seguras entre distintos dominios y equipos.

Anteriormente trabajó como Data Engineer en Tekne, donde desarrolló y orquestó procesos de ingesta y transformación en PySpark y Apache Airflow, contribuyendo a la generación de insights para clientes internacionales. En ese rol participó en proyectos de reporting y automatización, incluyendo soluciones orientadas a analítica y visualización en Power BI. Se destaca por su capacidad para trabajar en equipos multiculturales y bilingües, utilizando inglés de manera cotidiana en documentación técnica, reuniones y comunicación con clientes. Es Analista de Sistemas egresado del Instituto Tecnológico ORT y estudiante de Licenciatura en Tecnología de la Información en la Universidad de Palermo. Posee experiencia práctica en Python, PySpark, SQL, Databricks, Azure Data Factory y herramientas de CI/CD. Su Trabajo Final de Grado se centra en el problema del data lineage estático a nivel columna, proponiendo un enfoque híbrido que combina parsing determinístico e inteligencia artificial para reconstruir automáticamente el linaje de datos a partir de código fuente y mantenerlo actualizado mediante integración continua con repositorios Git. Interesado en la intersección entre Data Engineering, inteligencia artificial y finanzas, orienta su desarrollo profesional hacia liderazgo técnico y diseño de soluciones de datos e IA con impacto en productos y servicios B2B.